# A2 Medallion architecture

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

## Contents

**Track A: Databricks pipeline engineering ..... 2**
A2 Medallion architecture ..... 3
A2-L1 Bronze: keep every delivery exactly as received ..... 3
A2-L2 Silver: canonical names, checked types, one row per respondent per wave ..... 8
A2-L3 Gold: state the grain, group by all of it, guard every ratio ..... 12
Track A best-practice checklist ..... 18
**Appendix: solutions and answer keys ..... 19**

# Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

# A2 Medallion architecture

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Bronze, silver and gold contracts.

## A2-L1 Bronze: keep every delivery exactly as received

### Concept

Bronze is your evidence. Its contract has four rules.

1. **Append only.** Every delivery adds rows. Nothing in bronze is updated, deleted or overwritten. A redelivery of wave W14 appends again, with a new `ingestion_id`.
2. **As received.** No cleaning, filtering, renaming or type casting. Read CSV columns as strings. `inferSchema` guesses types, and a guess can destroy data: the ID `00123` becomes the integer `123`, and the leading zeros are gone for good.
3. **Audit columns.** Add four columns to every row: `ingestion_id` (one UUID per load), `source_file`, `ingestion_timestamp` and `delivery_version`. Later layers use them to pick the newest version and to trace any number back to a file.
4. **Grain.** One row per respondent per wave per delivery file. Duplicates across deliveries are expected here. Silver resolves them.

Why keep the mess? Because silver logic changes. When you fix a mapping bug in March, you rebuild silver from bronze for every wave since January. You can only do that if bronze still holds the original deliveries.

Two practical points for Databricks Runtime 17.3 LTS:

- Count what you wrote from the Delta history instead of calling `count()` before the write. `DeltaTable.forName(spark, table).history(1)` returns `operationMetrics["numOutputRows"]`. A separate `count()` reads the source file a second time.
- Open-source Spark defaults `saveAsTable` to Parquet. Databricks defaults to Delta. Write `.format("delta")` explicitly, so the code behaves the same in local tests.

## Why it matters

A bronze job reads supplier CSVs with `inferSchema=true`. Respondent IDs such as `00123` land as the integer `123`. Months later, the supplier sends a correction file keyed on the original IDs, and 18% of rows fail to match silver. The original IDs are not recoverable from bronze, because bronze never held them. The team asks the supplier to resend six months of deliveries.

Reading every column as a string keeps `00123` intact. Silver can then cast deliberately, with checks.

## Worked example: weak code

```
1 def ingest(path, table):
1     df = spark.read.option("header", True).option("inferSchema",
      True).csv(path)
2     df = df.filter(F.col("final_weight").isNotNull())
3     print(df.count())
4     df.write.mode("overwrite").saveAsTable(table)
```

1. **Line 2.** Type guessing. `00123` becomes `123`, and the original value is lost.
2. **Line 3.** Cleaning in bronze. The rows with missing weights vanish before anyone can count them.
3. **Line 4.** Reads the whole file a second time, only to print a number.
4. **Line 5.** Overwrite destroys every earlier delivery.

## Worked example: strong code

```
 1 import uuid
 2
 3 from delta.tables import DeltaTable
 4 from pyspark.sql import DataFrame, SparkSession
 5 from pyspark.sql import functions as F
 6
 7
 1 def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str,
     delivery_version: int) -> DataFrame:
 9     """Add the four bronze audit columns."""
10     return df.withColumns({
11         "ingestion_id": F.lit(ingestion_id),
12         "source_file": F.lit(source_file),
2         "ingestion_timestamp": F.current_timestamp(),
14         "delivery_version": F.lit(delivery_version),
15     })
16
17
18 def ingest_to_bronze(spark: SparkSession, path: str, table: str,
     delivery_version: int) -> int:
19     """Append one delivery file to bronze as received. Returns rows
     written."""
```

```
    raw = spark.read.option("header", True).csv(path)
    audited = add_audit_columns(raw, str(uuid.uuid4()), path,
delivery_version)
    audited.write.format("delta").mode("append").saveAsTable(table)
    metrics = DeltaTable.forName(spark, table).history(1).collect()[0]
["operationMetrics"]
    return int(metrics["numOutputRows"])
```

1. **Line 8.** A pure transform. Test it with two rows and no files.
2. **Line 13.** When the row landed, from the cluster clock.
3. **Line 20.** No inferSchema: every column arrives as a string, exactly as delivered.
4. **Line 22.** Append only, and Delta stated explicitly.
5. **Line 23.** The row count comes from the commit metrics. No second read of the file.

The strong version keeps every byte the supplier sent, labels every row with its origin, and reads the file once.

## Recap card

- **The four rules of the bronze contract** Append only. As received. Audit columns. Grain: respondent per wave per delivery file.
- **The four bronze audit columns** ingestion_id, source_file, ingestion_timestamp, delivery_version.
- **Why no inferSchema in bronze?** It guesses types and can destroy values, such as the leading zeros in 00123.
- **Rows written, without a second read** DeltaTable.forName(spark, t).history(1) and operationMetrics["numOutputRows"].

## Practice

### Exercise 1. Multiple choice (level 1)

A delivery contains 12 rows with a null `final_weight`. What should the bronze job do with them?

- a) Drop them, so silver never sees bad rows.
- b) Append them as received, with the audit columns.
- c) Replace the null weights with 1.0.
- d) Reject the whole file.

**Why?**

- r1) Bronze records deliveries as received. Silver and the data contract decide what to do with bad rows.
- r2) Delta tables cannot hold null values.

### Exercise 2. Predict the output (level 2)

`ids.csv` holds a header line `respondent_id` and one data line `00123`. What does this print on Spark 4.0?

```
df = spark.read.option("header", True).option("inferSchema", True).csv("ids.csv")
print(df.collect()[0]["respondent_id"])
```

### Exercise 3. Fill the gap (level 2)

Complete the bronze write and the timestamp audit column.

#### What you need to know.

`df.withColumn(name, column)` returns a new DataFrame with one column added, or replaced when the name exists. It is the one-column form of `withColumns({...})` from the worked example.

```
dated = df.withColumn("loaded_on", F.current_date())
```

The writer's `.mode(...)` decides what happens to rows already in the table: `"append"` adds the new rows, `"overwrite"` replaces the old ones.

```
audited = raw.withColumn("ingestion_timestamp", F._____(1)())
audited.write.format("delta").mode("_____(2)").saveAsTable(table)
```

### Exercise 4. Write the function (level 3)

Write `add_audit_columns(df, ingestion_id, source_file, delivery_version)`. It returns `df` with four extra columns: `ingestion_id`, `source_file`, `ingestion_timestamp` (current timestamp) and `delivery_version`. Do not change any existing column. Use type hints and a docstring.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Add the four bronze audit columns to a delivery.
def add_audit_columns(
    df: DataFrame,
    ingestion_id: str,
    source_file: str,
    delivery_version: int,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str, delivery_version: int) -> DataFrame:
    ...
```

### Exercise 5. Spot the bug (level 4)

Run this job twice for two deliveries of the same wave. Bronze ends with only the second delivery. Tap the line, then fix the function.

```
1 def ingest_to_bronze(spark: SparkSession, path: str, table: str,
    delivery_version: int) -> None:
2     """Append one delivery file to bronze as received."""
3     raw = spark.read.option("header", True).csv(path)
4     audited = add_audit_columns(raw, str(uuid.uuid4()), path,
    delivery_version)
5     audited.write.format("delta").mode("overwrite").saveAsTable(table)
```

Write down the line number of the bug. Then write the fix.

### Exercise 6. Architecture design (level 5)

Assign each piece of work to the layer that owns it.

Options: Bronze, Silver, Gold.

1. Store the supplier's CSV with every column as a string
2. Rename wgt to final_weight and cast it to DOUBLE
3. Keep only the newest delivery version per respondent and wave
4. Weighted reach by wave, age group and region for Power BI

## Practical: Build a bronze loader and prove it keeps every delivery

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds two small deliveries of the same wave in memory, loads both into bronze, and shows them side by side. In production the rows come from `spark.read.option("header", True).csv(path)`.

Download the notebook `A2-L1.ipynb` from the app.

## Sources

- inferSchema turns 00123 into the integer 123. (verified: Run on PySpark 4.0.4 in this repository.)
- Delta history operationMetrics holds numOutputRows for an append. (verified: Run on delta-spark 4.0.1 in this repository.)
- DataFrame.withColumns exists (PySpark 3.3 and later). (verified: Checked on PySpark 4.0.4 in this repository.)
- Open-source Spark defaults saveAsTable to Parquet; Databricks defaults to Delta. (verified: OSS behaviour run locally; Databricks default per docs (VERSIONS.md).)

## A2-L2 Silver: canonical names, checked types, one row per respondent per wave

### Concept

Silver turns deliveries into a dependable table. Its contract:

- **Canonical names.** Rename supplier columns to the names in the Variable Registry, loaded at run time.
- **Declared types.** Cast each column to the type in the registry.
- **Grain.** One row per respondent per wave. Keep the newest delivery version per key, then MERGE (lesson A3-L1).

Two Spark behaviours make the first two steps fail silently or loudly, in the wrong place.

**Renames skip missing columns.** `withColumnRenamed("wgt", "final_weight")` does nothing when `wgt` is absent, and so does `withColumnsRenamed`. No error, no warning. If the supplier renames `wgt` to `weight`, silver loses its weight column, and the failure surfaces hours later in gold. Check the source columns yourself before you rename, and raise one error that lists every missing column.

**Casts under ANSI mode.** Databricks Runtime 17.3 LTS runs Spark 4.0, which turns ANSI mode on by default. `cast("abc" as int)` now raises `CAST_INVALID_INPUT` and stops the job at the first bad value. Older runtimes returned NULL without a word. Neither is what you want in silver. Use `try_cast`, which returns NULL for a bad value, then count the new NULLs and let a data-quality check decide: under 0.1% is a WARN, above it is CRITICAL.

```
1 F.col("final_weight").try_cast("double")
```

The pattern is: check columns, rename, `try_cast`, count cast failures, deduplicate, MERGE. Each step is a small function you can test.

### Why it matters

A supplier renames `wgt` to `weight` without notice. The silver job calls `withColumnRenamed("wgt", "final_weight")`, which silently does nothing. The type-casting loop skips `final_weight` because the column is missing. Silver writes the wave with no weight column, since schema evolution is on. At 03:00 the gold job fails with `final_weight` not found. The morning report shows nothing for the new wave.

A column check before the rename fails the silver job at 01:00 with one clear message: "missing source columns: wgt". The on-call engineer knows exactly what to ask the supplier.

## Worked example: weak code

```
1  def transform_to_silver(bronze_df, column_map, type_map, wave_id):
2      for src, tgt in column_map.items():
1          bronze_df = bronze_df.withColumnRenamed(src, tgt)
4      for col_name, col_type in type_map.items():
2          bronze_df = bronze_df.withColumn(col_name,
       col(col_name).cast(col_type))
3      bronze_df = bronze_df.withColumn("wave_id", lit(wave_id))
7      return bronze_df
```

1. **Line 3.** A missing source column is skipped without an error.
2. **Line 5.** Under ANSI mode one bad value stops the job. Without ANSI, bad values become NULL with no count.
3. **Line 6.** Overwrites the wave_id from the delivery, hiding a file loaded against the wrong wave.

## Worked example: strong code

```
1  def require_columns(df: DataFrame, columns: list[str]) -> None:
2      """Raise ValueError listing every column missing from df."""
3      missing = sorted(set(columns) - set(df.columns))
4      if missing:
1          raise ValueError(f"missing source columns: {missing}")
6
7
8  def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) ->
     DataFrame:
9      """Rename supplier columns to registry names. Fails on any missing
       column."""
10     require_columns(df, list(column_map))
2      return df.withColumnsRenamed(column_map)
12
13
14 def cast_to_registry_types(df: DataFrame, type_map: dict[str, str]) ->
     DataFrame:
15
       """Cast with try_cast, so a bad value becomes NULL instead of stopping
       the job."""
16     require_columns(df, list(type_map))
3      return df.withColumns({c: F.col(c).try_cast(t) for c, t in
       type_map.items()})
18
19
20 def count_cast_failures(df: DataFrame, type_map: dict[str, str]) ->
     dict[str, int]:
```

```
21
    """Per column, rows whose non-null value fails try_cast. One pass over
    df."""
22      failures = [
4           F.count_if(F.col(c).isNotNull() &
    F.col(c).try_cast(t).isNull()).alias(c)
24          for c, t in type_map.items()
25      ]
26      return df.agg(*failures).collect()[0].asDict()
```

1. **Line 5.** One error that names every missing column.
2. **Line 11.** Safe to call now: every source column exists.
3. **Line 17.** Column.try_cast (PySpark 4.0) returns NULL for a bad value.
4. **Line 23.** Run on the uncast frame: a value that is present but fails the cast is a failure. All columns counted in one agg pass.

The strong version fails early and clearly on a changed delivery, and turns bad values into a number you can check instead of a crash or a silent NULL.

## Recap card

- **What does withColumnRenamed do with a missing column?** Nothing, silently. Check the source columns yourself and raise one error listing every missing one.
- **cast on Spark 4.0 (Runtime 17.3 LTS) with a malformed value** Raises CAST_INVALID_INPUT, because ANSI mode is on by default.
- **Safe cast for silver** F.col(c).try_cast(t), then count the new NULLs and check them against a threshold.
- **Silver dedup key** The full grain: respondent_id and wave_id.

## Practice

### Exercise 1. Predict the output (level 1)

What does this print on PySpark 4.0?

**What you need to know.**

`spark.createDataFrame(rows, schema)` builds a small DataFrame from Python tuples, one tuple per row. The schema string names each column and gives its type. Tests and examples use it to make data without a table.

`df.columns` returns the column names as a Python list, in order.

```
1 df = spark.createDataFrame([("R1", 3)], "id STRING, n INT")
2 df.columns   # ['id', 'n']
```

```
1 df = spark.createDataFrame([(1, 2.5)], "respondent_id INT, weight DOUBLE")
2 print(df.withColumnRenamed("wgt", "final_weight").columns)
```

### Exercise 2. Multiple choice (level 2)

On Databricks Runtime 17.3 LTS, silver runs `F.col("final_weight").cast("double")`. One row holds `"n/a"`. What happens?

- a) The row gets NULL in final_weight, and the job succeeds.
- b) The job fails with CAST_INVALID_INPUT.
- c) The row is dropped.
- d) The value becomes 0.0.

**Why?**

- r1) Spark 4.0 enables ANSI mode by default, and ANSI casts raise on malformed input.
- r2) Delta rejects NULL values in DOUBLE columns.

### Exercise 3. Fill the gap (level 2)

Cast every column in `type_map` so that a bad value becomes NULL instead of stopping the job.

```
df = df.withColumns({c: F.col(c)._____(1)(t) for c, t in type_map.items()})
```

### Exercise 4. Write the function (level 3)

Write `rename_to_canonical(df, column_map)`. It raises `ValueError` naming every missing source column, then renames all columns in one call. Use type hints and a docstring.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Rename supplier columns to registry names.
def rename_to_canonical(
    df: DataFrame,
    column_map: dict[str, str],
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame


def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) -> DataFrame:
    ...
```

### Exercise 5. Spot the bug (level 4)

Silver's grain is one row per respondent per wave. After loading waves W1 and W2, silver holds only one row per respondent. Tap the line, then fix the function.

```
1 def prepare_silver(df: DataFrame, column_map: dict[str, str], type_map:
    dict[str, str]) -> DataFrame:
2     """Rename, cast and keep the newest version per respondent and
    wave."""
3     renamed = rename_to_canonical(df, column_map)
4     typed = cast_to_registry_types(renamed, type_map)
5     return latest_per_key(typed, ["respondent_id"], "delivery_version")
```

Write down the line number of the bug. Then write the fix.

### Exercise 6. Put in order (level 5)

Put the silver steps in order.

- Count cast failures and apply the WARN or CRITICAL threshold
- Rename to canonical names
- Check that every mapped source column exists
- MERGE into silver
- Keep the newest version per respondent and wave
- try_cast to registry types

### Sources

- withColumnRenamed and withColumnsRenamed ignore missing columns without an error. (verified: Run on PySpark 4.0.4 in this repository.)
- Spark 4.0 enables ANSI mode by default; cast raises CAST_INVALID_INPUT on malformed input; try_cast returns NULL. (verified: Run on PySpark 4.0.4 in this repository. Databricks Runtime 17.3 LTS uses Spark 4.0.0 (search-only, VERSIONS.md).)
- Column.try_cast exists in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)

## A2-L3 Gold: state the grain, group by all of it, guard every ratio

### Concept

Gold tables feed Power BI and clients. The contract has three parts.

**1. State the grain.** Write it in the function docstring and on the table. "One row per tracker x wave x age group x gender x region." Store it as a table comment or a custom table property:

```
COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x
    age_group x gender x region';
ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' =
    'tracker x wave x age_group x gender x region');
```

Avoid the property name `owner`. It is reserved, and Spark 4.0 rejects it with `UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY`. Ownership is set with `ALTER TABLE ... OWNER TO`.

**2. Group by every grain column.** If the grain names `tracker_id`, the `groupBy` must include `tracker_id`. Miss one and two trackers' respondents fall into the same row. The numbers look plausible and are wrong.

**3. Make aggregates safe.**

- A `sum` over no matching rows returns NULL, not 0. `F.sum(F.when(F.col("reach_flag") == 1, 1))` gives NULL for a subgroup with no reach. Wrap it: `F.coalesce(F.sum(...), F.lit(0))`.
- Guard every ratio. `F.when(F.col("weighted_base") > 0, reach / base)` returns NULL for an empty base instead of an error or an infinite value.
- Keep the unweighted count next to every weighted figure. Analysts need it to judge whether a subgroup is too small to report.

Gold is also where you check totals against silver. The sum of `weighted_base` across all subgroups for a wave must equal the silver weight total for in-scope respondents. Lesson A6-L3 builds that check.

## Why it matters

The reference `compute_gold_reach_metrics` promises "tracker x wave x subgroup grain" but groups by wave and subgroup only. When a second tracker starts writing to the same silver table, its respondents fold into the first tracker's rows. Weighted bases roughly double, and reach percentages blend two audiences. The dashboard shows smooth, believable figures for three weeks, until a client compares a subgroup against its published sample size.

Adding `tracker_id` to `groupBy`, and a test that checks the output has one row per grain key, prevents it.

## Worked example: weak code

```
from pyspark.sql.functions import col, count, sum, when
```

```
2
3  def compute_gold_reach_metrics(silver_table, wave_id,
       weight_col="final_weight"):
4      """Compute reach metrics at tracker x wave x subgroup grain."""
5      silver = spark.table(silver_table).filter(col("wave_id") == wave_id)
6      return (
7          silver.filter(col("in_scope") == True)
2          .groupBy("wave_id", "age_group", "gender", "region")
9          .agg(
10             count("*").alias("unweighted_n"),
11             sum(weight_col).alias("weighted_base"),
3              sum(when(col("reach_flag") == 1,
       col(weight_col))).alias("weighted_reach"),
13         )
4          .withColumn("weighted_reach_pct", col("weighted_reach") /
       col("weighted_base"))
15     )
```

1. **Line 1.** Imports `sum` by name, shadowing the Python built-in (reference defect 3).
2. **Line 8.** The docstring promises tracker grain, but tracker_id is missing.
3. **Line 12.** NULL, not 0, for a subgroup where nobody is reached.
4. **Line 14.** No guard on the base, and NULL reach gives a NULL percentage.

## Worked example: strong code

```
1  from pyspark.sql import DataFrame
2  from pyspark.sql import functions as F
3
1  GRAIN = ["tracker_id", "wave_id", "age_group", "gender", "region"]
5
6
7  def reach_metrics(df: DataFrame, weight_col: str = "final_weight") ->
       DataFrame:
8      """Weighted reach at tracker x wave x age_group x gender x region
       grain.
9
10     Args:
11         df: In-scope silver respondents with reach_flag and weights.
12         weight_col: Weight column to aggregate.
13
14     Returns:
15         One row per GRAIN key with unweighted_n, weighted_base,
       weighted_reach
16         and weighted_reach_pct (NULL when the base is 0).
17     """
18     reached = F.when(F.col("reach_flag") == 1, F.col(weight_col))
2      metrics = df.groupBy(*GRAIN).agg(
20         F.count("*").alias("unweighted_n"),
21         F.coalesce(F.sum(weight_col), F.lit(0.0)).alias("weighted_base"),
3          F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach"),
23     )
```

```
24      return metrics.withColumn(
25          "weighted_reach_pct",
4           F.when(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base")),
27      )
```

1. **Line 4.** The grain is one constant, used by the groupBy and by the tests.
2. **Line 19.** Groups by every grain column, tracker included.
3. **Line 22.** Zero, not NULL, when a subgroup has no reach.
4. **Line 26.** NULL for an empty base, never an error or infinity.

The strong version cannot merge two trackers, reports a true zero for zero reach, and never divides by nothing.

## Recap card

- **Where do you record a gold table's grain?** In the function docstring and on the table: COMMENT ON TABLE or a custom property such as 'grain'.
- **sum over no matching rows** Returns NULL, not 0. Wrap it: F.coalesce(F.sum(...), F.lit(0)).
- **Safe ratio in PySpark** F.when(base > 0, part / base). NULL when the base is 0.
- **Why is SET TBLPROPERTIES ('owner' = ...) wrong?** owner is reserved. Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. Use ALTER TABLE ... OWNER TO.

## Practice

### Exercise 1. Spot the bug (level 1)

The docstring states the grain. The code does not match it. Tap the line, then fix the function.

```
1  def base_by_subgroup(df: DataFrame) -> DataFrame:
2      """Weighted base at tracker x wave x region grain."""
3      return df.groupBy("wave_id",
       "region").agg(F.sum("final_weight").alias("weighted_base"))
```

Write down the line number of the bug. Then write the fix.

### Exercise 2. Predict the output (level 2)

Region `S` has one respondent, and nobody in `S` is reached. What does this print?

```
1  df = spark.createDataFrame([("S", 0)], "region STRING, reach_flag INT")
```

```
out = df.groupBy("region").agg(F.sum(F.when(F.col("reach_flag") == 1,
    1)).alias("reached"))
print(out.collect()[0]["reached"])
```

### Exercise 3. Fill the gap (level 3)

Make reach a true zero when nobody is reached, and the percentage NULL when the base is zero.

```
F._____(1)(F.sum(reached), F.lit(0.0)).alias("weighted_reach")
...
F._____(2)(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base"))
```

### Exercise 4. Write the function (level 4)

Write `reach_metrics(df, grain)`. It groups by every column in `grain` and returns `unweighted_n`, `weighted_base` (sum of `final_weight`, 0 when empty), `weighted_reach` (sum of `final_weight` where `reach_flag == 1`, 0 when none) and `weighted_reach_pct` (NULL when the base is 0).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted reach, one row per grain key.
def reach_metrics(
    df: DataFrame,
    grain: list[str],
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def reach_metrics(df: DataFrame, grain: list[str]) -> DataFrame:
    ...
```

### Exercise 5. Multiple choice (level 5)

You want the grain of `research_gold.nat_metrics` recorded on the table itself. Which statement fails on Spark 4.0?

- a) `COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x region'`
- b) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' = 'tracker x wave x region')`
- c) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('owner' = 'data.engineering@example.org')`

- d) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('layer' = 'gold')`

**Why?**

- r1) `owner` is a reserved table property, so Spark rejects setting it this way.
- r2) Table properties cannot hold email addresses.

## Sources

- sum(when(cond, 1)) over rows where cond is false returns NULL. (verified: Run on PySpark 4.0.4 in this repository.)
- 'owner' is a reserved table property; Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. (verified: Run on PySpark 4.0.4 with delta-spark 4.0.1 in this repository.)
- COMMENT ON TABLE and custom TBLPROPERTIES work on Delta tables. (verified: Run on delta-spark 4.0.1 in this repository.)

## Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

# Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

## A2-L1 Bronze: keep every delivery exactly as received

### A2-L1 exercise 1

**Hints.** (1) What is bronze for? (2) Which layer cleans and validates? (3) Bronze keeps everything, labelled with where it came from.

**Answer.** b) Append them as received, with the audit columns. Because: r1) Bronze records deliveries as received. Silver and the data contract decide what to do with bad rows.

Bronze records every delivery as received. The contract check flags the null weights, and silver decides how to handle them. Nothing is lost.

**Common wrong answers.**

- `a`: Dropping them in bronze destroys the evidence. You can no longer tell the supplier which rows were wrong.
- `c`: Imputing a weight in bronze invents data. The weighted totals would look plausible and be wrong.
- `d`: Accept or reject is the contract check's decision, after the delivery is recorded. Bronze records first.
- `b|r2`: Right action, wrong reason. Delta columns accept nulls unless you add a NOT NULL constraint.

### A2-L1 exercise 2

**Hints.** (1) What type does inferSchema pick for a column of digits? (2) Integers have no leading zeros. (3) The value prints as an int.

**Answer.**

```
1 123
```

`inferSchema` reads the column as an integer, so `00123` becomes `123`. Read bronze without inferSchema and every column stays a string, zeros included.

**Common wrong answers.**

- `00123`: That is what you get without inferSchema. With it, Spark sees digits, picks an integer type, and drops the leading zeros.

- `'00123'`: The value is no longer a string. inferSchema made the column an integer.

### A2-L1 exercise 3

**Hints.** (1) Bronze never removes what is already there. (2) The timestamp function's name describes the current moment. (3) `current_timestamp` and `append`.

**Answer.**

```
audited = raw.withColumn("ingestion_timestamp", F.current_timestamp())
audited.write.format("delta").mode("append").saveAsTable(table)
```

`F.current_timestamp()` stamps each load. `mode("append")` adds rows and leaves earlier deliveries untouched.

**Common wrong answers.**

- g2 = `overwrite`: Overwrite deletes every earlier delivery. Bronze is append-only.
- g1 = `now`: There is no `F.now`. The PySpark function is `F.current_timestamp()`.
- g2 = `merge`: `merge` is not a write mode, and bronze never updates rows. Use `append`.

### A2-L1 exercise 4

**Hints.** (1) `df.withColumns({...})` adds several columns in one call (PySpark 3.3 and later). (2) Constants go through `F.lit()`. The timestamp comes from `F.current_timestamp()`. (3) Add columns only. No filter, no cast.

**Answer.**

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def add_audit_columns(df: DataFrame, ingestion_id: str, source_file: str, delivery_version: int) -> DataFrame:
    """Add the four bronze audit columns to a delivery.

    Args:
        df: The delivery as read, all columns as strings.
        ingestion_id: One UUID for this load.
        source_file: Path of the delivery file.
        delivery_version: Supplier's version number for the delivery.

    Returns:
        The delivery with ingestion_id, source_file, ingestion_timestamp and delivery_version.
```

```
    """
    return df.withColumns({
        "ingestion_id": F.lit(ingestion_id),
        "source_file": F.lit(source_file),
        "ingestion_timestamp": F.current_timestamp(),
        "delivery_version": F.lit(delivery_version),
    })
```

`withColumns` adds all four audit columns in one step. Everything else stays as delivered: no filter, no cast.

### A2-L1 exercise 5

**Hints.** (1) What happens to existing rows when this job writes? (2) One word on line 5 decides it. (3) Use `mode("append")`.

**Answer.** The bug is on line 5.

Fixed version:

```
1 def ingest_to_bronze(spark: SparkSession, path: str, table: str,
    delivery_version: int) -> None:
2     """Append one delivery file to bronze as received."""
3     raw = spark.read.option("header", True).csv(path)
4     audited = add_audit_columns(raw, str(uuid.uuid4()), path,
    delivery_version)
5     audited.write.format("delta").mode("append").saveAsTable(table)
```

`overwrite` replaces the table, so each delivery erases the one before. Bronze must append.

**Common wrong answers.**

- Line 3: Reading without inferSchema is correct for bronze. Look at how the rows are written.
- Line 4: A new UUID per load is correct: each delivery gets its own ingestion_id.

### A2-L1 exercise 6

**Hints.** (1) Bronze never changes what the supplier sent. (2) Silver has one clean row per respondent per wave. (3) Gold holds aggregates.

**Answer.**

- Bronze stores deliveries as received.
- Silver renames, casts and resolves versions, so it holds one row per respondent per wave.
- Gold aggregates for reporting, with the grain stated.

Bronze keeps evidence, silver makes it clean and unique, gold makes it ready to report.

**Common wrong answers.**

- cast = `bronze`: Casting in bronze loses the original value when the cast fails or guesses wrong. Cast in silver, with checks.
- dedupe = `bronze`: Bronze keeps every delivery, duplicates included. Silver picks the newest version.
- reach = `silver`: Silver holds respondent-level rows. Aggregates for reporting belong in gold.

## A2-L2 Silver: canonical names, checked types, one row per respondent per wave

### A2-L2 exercise 1

**Hints.** (1) Does the frame have a column called `wgt`? (2) What does withColumnRenamed do when the source column is missing? (3) It returns the frame unchanged.

**Answer.**

```
1 ['respondent_id', 'weight']
```

`withColumnRenamed` is a no-op when the source column is missing. No exception, no warning. Check the columns yourself before you rename.

**Common wrong answers.**

- `['respondent_id', 'final_weight']`: There is no column called `wgt`, so nothing is renamed. `weight` keeps its name.
- `AnalysisException`: No error. withColumnRenamed silently ignores a missing column, which is exactly the danger.

### A2-L2 exercise 2

**Hints.** (1) Which Spark version does Runtime 17.3 LTS use? (2) What changed about casts in Spark 4.0? (3) ANSI mode is on by default.

**Answer.** b) The job fails with CAST_INVALID_INPUT. Because: r1) Spark 4.0 enables ANSI mode by default, and ANSI casts raise on malformed input.

Spark 4.0 turns ANSI mode on, so a malformed cast raises `CAST_INVALID_INPUT`. `try_cast` returns NULL instead, which you then count.

**Common wrong answers.**

- `a`: That was the behaviour before ANSI mode became the default. On Spark 4.0 use `try_cast` to get NULL.
- `c`: Casting never drops rows. It either raises or returns a value.

- `d`: Spark never turns malformed text into 0.0. It raises, or with try_cast returns NULL.
- `b|r2`: Right outcome, wrong reason. Delta accepts NULLs unless a NOT NULL constraint says otherwise. The error comes from the ANSI cast.

### A2-L2 exercise 3

**Hints.** (1) The safe variant of cast starts with a word meaning attempt. (2) It exists as a SQL function and, since PySpark 4.0, as a Column method. (3) `try_cast`.

**Answer.**

```
df = df.withColumns({c: F.col(c).try_cast(t) for c, t in
    type_map.items()})
```

`Column.try_cast` returns NULL where the value cannot be cast. Count those NULLs and let a data-quality check decide what to do.

**Common wrong answers.**

- g1 = `cast`: Under ANSI mode on Spark 4.0, `cast` raises on the first bad value. You need the variant that returns NULL.
- g1 = `astype`: `astype` is an alias of `cast`, so it raises under ANSI mode too.

### A2-L2 exercise 4

**Hints.** (1) Find the missing columns with a set difference. (2) `missing = sorted(set(column_map) - set(df.columns))`. (3) Raise if `missing` is not empty, then `return df.withColumnsRenamed(column_map)`.

**Answer.**

```
from pyspark.sql import DataFrame


def rename_to_canonical(df: DataFrame, column_map: dict[str, str]) ->
   DataFrame:
    """Rename supplier columns to registry names.

    Args:
        df: A bronze delivery.
        column_map: Supplier name to canonical name.

    Returns:
        The frame with canonical names.

    Raises:
        ValueError: If any source column in column_map is missing from df.
    """
    missing = sorted(set(column_map) - set(df.columns))
```

```
    if missing:
        raise ValueError(f"missing source columns: {missing}")
    return df.withColumnsRenamed(column_map)
```

The set difference lists every missing source column in one go. Only then is the rename safe, because `withColumnsRenamed` would skip missing columns without telling you.

### A2-L2 exercise 5

**Hints.** (1) What is silver's grain? (2) Which columns does the deduplication treat as the key? (3) The key must include wave_id.

**Answer.** The bug is on line 5.

Fixed version:

```
def prepare_silver(df: DataFrame, column_map: dict[str, str], type_map: dict[str, str]) -> DataFrame:
  """Rename, cast and keep the newest version per respondent and wave."""
  renamed = rename_to_canonical(df, column_map)
  typed = cast_to_registry_types(renamed, type_map)
  return latest_per_key(typed, ["respondent_id", "wave_id"], "delivery_version")
```

Deduplicating on `respondent_id` alone keeps one wave per respondent and throws the others away. The key must match the grain: respondent and wave.

**Common wrong answers.**

- Line 3: The rename is checked and correct. Look at which keys the deduplication uses.
- Line 4: try_cast is the right choice for silver. The rows vanish in a later step.

### A2-L2 exercise 6

**Hints.** (1) Checks on source names come before anything renames them. (2) A CRITICAL result must stop the run before the write. (3) The MERGE needs a source with one row per key.

**Answer.**

1. Check that every mapped source column exists
2. Rename to canonical names
3. try_cast to registry types
4. Count cast failures and apply the WARN or CRITICAL threshold
5. Keep the newest version per respondent and wave

6. MERGE into silver

Check, rename, cast, count failures, deduplicate, merge. Each step depends on the one before, and nothing reaches silver until the checks pass.

**Common wrong answers.**

- After the rename, the source names are gone. Check the source columns first.
- Count cast failures before the MERGE. A CRITICAL failure must stop bad data reaching silver.
- Deduplicating after the MERGE is too late. A repeated key fails the MERGE, or inserts twice when the target has no match.

## A2-L3 Gold: state the grain, group by all of it, guard every ratio

### A2-L3 exercise 1

**Hints.** (1) List the grain columns in the docstring. (2) Compare them with the groupBy. (3) tracker_id is missing.

**Answer.** The bug is on line 3.

Fixed version:

```
1 def base_by_subgroup(df: DataFrame) -> DataFrame:
2     """Weighted base at tracker x wave x region grain."""
3     return df.groupBy("tracker_id", "wave_id",
    "region").agg(F.sum("final_weight").alias("weighted_base"))
```

The grain names three columns, and the `groupBy` has two. Without `tracker_id`, rows from different trackers fold into one.

**Common wrong answers.**

- Line 2: The docstring is the contract, and it is right. The code has to match it.

### A2-L3 exercise 2

**Hints.** (1) What does `F.when(cond, 1)` return when `cond` is false? (2) What is the sum of a column that holds only NULL? (3) NULL prints as None in Python.

**Answer.**

```
1 None
```

`F.when` without `otherwise` yields NULL for the unmatched row, and the sum of only NULLs is NULL. Power BI shows a blank instead of 0. Wrap the sum in `F.coalesce(..., F.lit(0))`.

**Common wrong answers.**

- `0`: `F.when` without `otherwise` returns NULL for non-matching rows, and `sum` of only NULLs is NULL. Wrap it in `F.coalesce(..., F.lit(0))`.
- `1`: The `when` condition is false for the only row, so there is nothing to add.

### A2-L3 exercise 3

**Hints.** (1) The first function returns its first non-null argument. (2) The second returns a value only when a condition holds. (3) `coalesce` and `when`.

**Answer.**

```
F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach")
...
F.when(F.col("weighted_base") > 0, F.col("weighted_reach") /
    F.col("weighted_base"))
```

`F.coalesce(x, 0)` turns a NULL sum into 0. `F.when(base > 0, ratio)` returns NULL when the base is 0, because there is no `otherwise`.

**Common wrong answers.**

- g1 = `ifnull`: `ifnull` works in SQL, but the standard DataFrame function for this is `F.coalesce`, which takes any number of columns.
- g2 = `where`: `where` filters rows. You want a conditional value per row: `F.when`.
- g1 = `nvl`: `nvl` is a SQL function name. In the DataFrame API use `F.coalesce`.

### A2-L3 exercise 4

**Hints.** (1) Build the reached-weight expression with `F.when`, then aggregate with `groupBy(*grain)`. (2) Wrap each sum in `F.coalesce(..., F.lit(0.0))`. (3) Add the percentage with `F.when(F.col("weighted_base") > 0, reach / base)`.

**Answer.**

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def reach_metrics(df: DataFrame, grain: list[str]) -> DataFrame:
    """Weighted reach, one row per grain key.

    Args:
        df: In-scope respondents with reach_flag and final_weight.
```

```
        grain: Columns that define one output row.

    Returns:
        unweighted_n, weighted_base, weighted_reach and weighted_reach_pct per grain key.
    """
    reached = F.when(F.col("reach_flag") == 1, F.col("final_weight"))
    metrics = df.groupBy(*grain).agg(
        F.count("*").alias("unweighted_n"),
        F.coalesce(F.sum("final_weight"), F.lit(0.0)).alias("weighted_base"),
        F.coalesce(F.sum(reached), F.lit(0.0)).alias("weighted_reach"),
    )
    return metrics.withColumn(
        "weighted_reach_pct",
        F.when(F.col("weighted_base") > 0, F.col("weighted_reach") / F.col("weighted_base")),
    )
```

`groupBy(*grain)` takes the grain as data, so the same function serves every gold table. Coalesce turns empty sums into 0, and the guarded ratio returns NULL for an empty base.

### A2-L3 exercise 5

**Hints.** (1) Three of these are ordinary custom properties or comments. (2) One property key has a special meaning in Spark. (3) Ownership is set with ALTER TABLE ... OWNER TO.

**Answer.** c) `ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('owner' = 'data.engineering@example.org')` Because: r1) `owner` is a reserved table property, so Spark rejects setting it this way.

Spark reserves `owner` and rejects it with `UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY`. Your reference `unity_catalog.sql` sets it, so that script fails. Use `ALTER TABLE ... OWNER TO` for ownership, and a comment or custom property for the grain.

**Common wrong answers.**

- `a`: COMMENT ON TABLE is valid and a good home for the grain.
- `b`: A custom property such as `grain` is valid. Tools can read it with SHOW TBLPROPERTIES.
- `d`: A custom property such as `layer` is valid.
- `c|r2`: Right statement, wrong reason. Property values can be any string. The key `owner` is reserved, and Spark raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY.